Oggetto Relazione progetto Programmazione a Oggetti 2024/2025

Alunno Murtezan Besnik, mat. 2113195

Progetto LIBZ

Introduzione

LIBZ è un'applicazione volta alla gestione di una libreria di contenuti multimediali che rende disponibili creazione, ricerca, modifica e cancellazione degli elementi della libreria.

In particolare si rende disponibile la gestione di elementi quali Libri, Film, Videogiochi, ognuno con attributi differenti.

Il principale punto di forza di LIBZ è la semplicità e la chiarezza dell'interfaccia grafica, volta a migliorare l'esperienza dell'utente per un uso piacevole ed intuitivo.

Ho scelto questo progetto poiché trovo soddisfacente organizzare elementi in un unico luogo migliorando un piccolo aspetto della vita dell'utente finale. Ho inoltre sfruttato la mia attenzione al dettaglio per creare una grafica pulita e gradevole.

Descrizione del modello

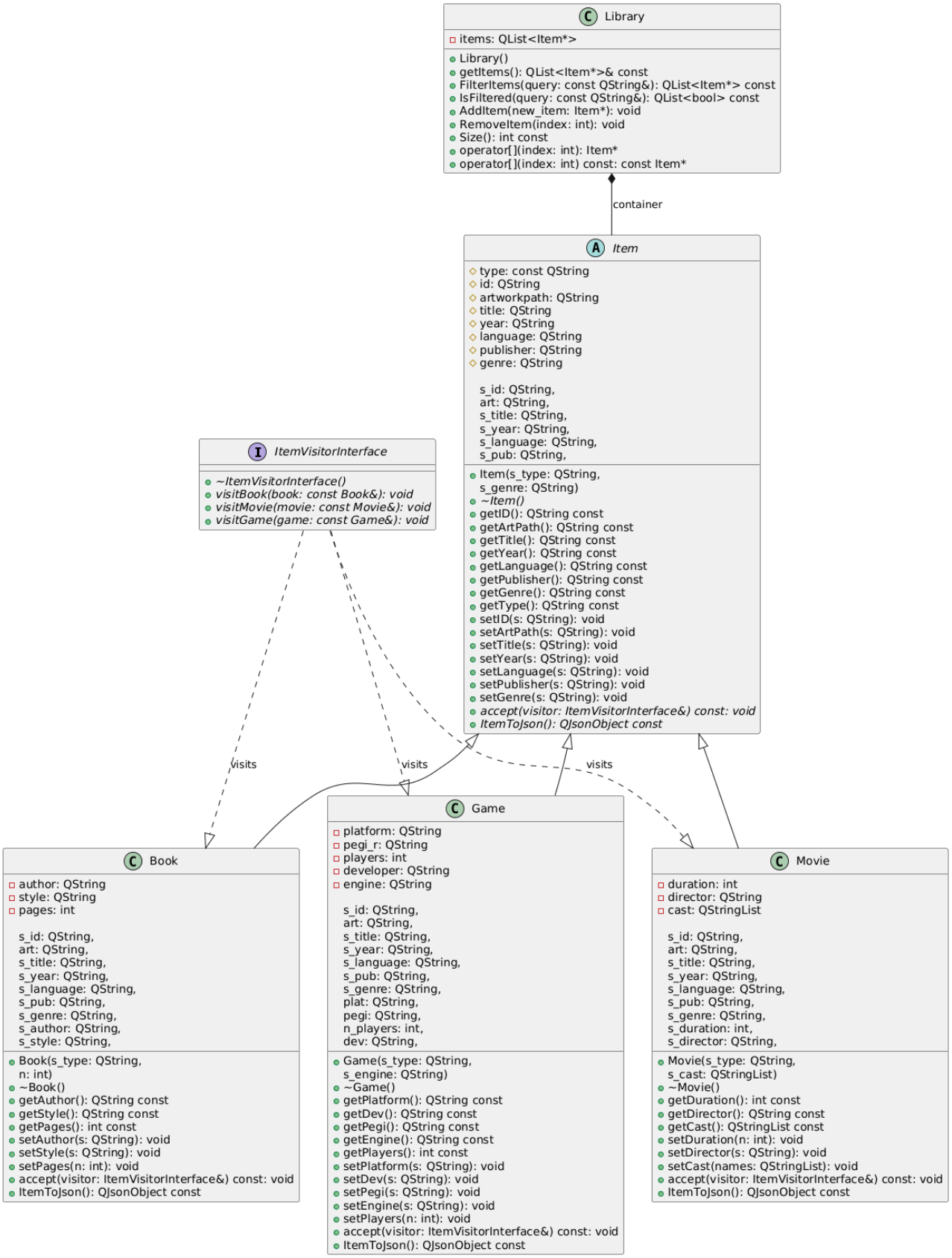
Il modello logico è composto da:

- **1 classe *base astratta* Item**: contiene le informazioni comuni (vedi schema UML; si fa notare che per scelta progettuale il membro *type* è di tipo costante in quanto non si vuole poter rendere disponibile la modifica del tipo una volta confermata la creazione dell'item) a tutti gli elementi della libreria con relativi metodi *Getters* e *Setters* e dichiara 2 metodi virtuali astratti *accept e ItemToJson* rispettivamente per implementare il pattern ***visitor*** e la conversione in formato JSON dei dati.
- **1° classe *concreta derivata* Book**: rappresenta un libro (aggiunge *Autore, Stile e N° Pagine*) ed implementa i 2 metodi virtuali astratti della classe base.
- **2° classe *concreta derivata* Movie**: rappresenta un film (aggiunge *Regista, Durata e Cast*) ed implementa i 2 metodi virtuali astratti della classe base.
- **3° classe *concreta derivata* Game**: rappresenta un videogioco (aggiunge *Piattaforma, PEGI Rating, Sviluppatore, N° Giocatori e Motore di Sviluppo*) ed implementa i 2 metodi virtuali astratti della classe base.
- **1 classe *contenitore* Library**: raccoglie l'insieme degli elementi della libreria.
- **1 interfaccia ItemVisitorInterface**: dichiara i metodi **visit*** che verranno poi implementati dalla relativa classe della GUI.

Inoltre sono state utilizzate altre classi di Qt quali QString e QList, oltre alle varie classi necessarie alla conversione dei dati in formato JSON (QJSONDocument etc.).

Viene utilizzato un namespace JSON_Utility (non presente nel diagramma) per la gestione di I/O.

(Di seguito è riportato il diagramma UML)



Polimorfismo

Il polimorfismo viene utilizzato per 2 aspetti:

1. ***Design Pattern Visitor*** : viene utilizzato per poter visualizzare le informazioni specifiche in base al tipo di elemento presente nella libreria. In base all'item selezionato, i *widget* volti alla visualizzazione dettagliata si aggiornano dinamicamente. A tale scopo si utilizza la classe **ItemInfoVisitor** che deriva ed implementa l'interfaccia ***ItemVisitorInterface*** e costruisce un *widget* ad hoc.

2. ***Conversione Item → JSON*** : per convertire in JSON i dati di ogni item viene utilizzata una *funzione virtuale ItemToJson* dichiarata nella classe base e successivamente implementata da ognuna delle classi derivate. La funzione utilizza le classi fornite da Qt per la gestione dei formati JSON quali QJSONArray, QJSONObject e QJSONValue. Per evitare di dover richiamare i metodi getter della classe Item all'interno del metodo delle classi concrete si sono posti i membri della classe base come protetti. Si fa notare che nel caso della classe **Movie** si ha un membro QStringList, il quale differenzia la struttura dell'output JSON di tale classe aggiungendo un array tra i valori. Con tale funzione polimorfa si evita di effettuare *dynamic_cast* per ottenere il tipo dinamico del puntatore all'interno della libreria.

Persistenza dei dati

Per la persistenza dei dati viene utilizzato il formato JSON, un unico file per la libreria di elementi costituito da un vettore. Gli elementi sono perlopiù semplici associazioni chiave-valore, al di fuori di quelli di tipo **Movie** che presentano al loro interno il vettore del *Cast*. Per l'Input si utilizza il namespace JSON_Utility il quale legge i dati dal formato JSON ed inserisce gli item nella libreria dopo averli allocati in memoria mentre per l'Output ci si avvale anche della funzione virtuale *ItemToJson* descritta in precedenza. Si rende disponibile un file di esempio "example.json" all'interno della cartella EXAMPLE_DATA contenente anche artwork extra nel caso si voglia testare l'aggiunta di nuovi elementi.

Funzionalità implementate

Le funzionalità implementate sono, per semplicità, suddivise in due categorie: funzionali ed estetiche.

Le prime comprendono:

- Gestione di 3 tipologie di elementi
- Conversione e salvataggio in formato JSON
- Funzionalità di ricerca completa su tutti i campi con possibilità di modifica o rimozione dell'elemento ricercato

Le funzionalità grafiche:

- Finestra di avvio con scelta di Creazione/Caricamento Libreria (in tal modo si evita la chiusura del programma alla chiusura della libreria corrente; nel caso si scelga di caricare una libreria ma si annulla la finestra di dialogo, verrà creata una libreria nuova)
- Pulsante di Salvataggio della Libreria (è necessario specificare sempre il percorso di destinazione)
- Pulsante di Chiusura della Libreria (non rileva modifiche non salvate)
- Barra di Ricerca in tempo reale
- Pulsanti di Aggiunta, Modifica, Rimozione
- Artwork di default automatico nel caso non si abbia una immagine disponibile
- Navigazione tra gli elementi tramite tasti frecce
- Gestione del ridimensionamento (è presente uno split verticale per ridimensionare la lista degli elementi in relazione alla visuale dettagliata)
- Ogni tipologia di prodotto ha una propria visualizzazione
- Possibilità di navigare attraverso diverse schermate tramite interfaccia
- Utilizzo di icone nei pulsanti
- Utilizzo di immagini nella visualizzazione degli oggetti
- Utilizzo di colori e stili grafici
- Effetti grafici come cambio del colore al passaggio del mouse
- Shortcut per Apertura Libreria (Ctrl+O), Salvataggio (Ctrl+S), Aggiunta Elementi (Ctrl+N)

Le funzionalità elencate sono intese in aggiunta a quanto richiesto dalle specifiche del progetto.

Rendicontazione ore

Attività	Ore Previste	Ore Effettive
Studio e progettazione	6	6
Sviluppo del codice del modello	6	6
Studio del framework Qt	10	11
Sviluppo del codice della GUI	8	9
Test e debug	8	10
Stesura della relazione	2	3
totale	40	**45**

Il monte ore è stato leggermente superato in quanto lo sviluppo del codice della GUI (in particolare la personalizzazione dei colori) e l'apprendimento del Framework Qt ha richiesto più tempo di quanto previsto, specialmente per eseguire test vari.